

Algoritmi-Modulo algoritmi per bioinformatica

Paradigma programmazione dinamica-Capitolo 14 CLRS

Roberto Posenato

versione 2026-2, 14/05/2026

- 1 Introduzione
 - Idea generale
- 2 Dettagli
- 3 Esempi di applicazione
 - Confronto di sequenze
 - Massima sottosequenza comune (LCS)
 - String matching approssimato
 - Problema dello Zaino
- 4 Casi comuni
- 5 Memoization
- 6 Esercizi

In questa lezione si introducono i concetti fondamentali del paradigma *programmazione dinamica* e si illustrano alcuni esempi di problemi risolti mediante questo paradigma.

Idea generale

- La tecnica della programmazione dinamica è stata introdotta da Bellman (1957) per progettare algoritmi che risolvono **problemi di ottimizzazione**.

La tecnica consiste di 4 fasi:

- 1 Caratterizzazione della struttura di una soluzione ottima.
- 2 Definizione ricorsiva del **valore** di una soluzione ottima.
- 3 Calcolo del valore di una soluzione ottima secondo uno schema bottom-up.
- 4 Costruzione della soluzione ottima dalle informazioni calcolate.

Idea generale

Programmazione dinamica in termini moderni

- La programmazione dinamica risolve un problema descrivendolo come un insieme di **stati** o **sottoproblemi**.
- Per ogni stato si definiscono:
 - i **casi base**;
 - le **transizioni**, cioè come combinare stati più piccoli per ottenere il valore dello stato corrente;
 - l'**ordine di calcolo**, che deve rispettare le dipendenze tra stati.
- Il costo dell'algoritmo è spesso:

$\# \text{stati} \times \text{costo per calcolare una transizione.}$

- La stessa ricorrenza può essere implementata con una tabella bottom-up oppure con ricorsione e memoization.

Idea generale

Confronto programmazione dinamica (tabulazione) – *divide et impera*

	tabulazione bottom-up	<i>divide et impera</i>
tipo risoluzione	bottom-up	top-down
tipo di programma	iterativa	ricorsiva
uso memoria	esplicito per i risultati	implicito
applicazione ideale	sottoistanze non disgiunte	sottoistanze disgiunte

La programmazione dinamica può essere implementata anche top-down usando memoization.

Programmazione dinamica

Quando si usa

Si usa quando:

- una soluzione ottima contiene al suo interno le soluzioni ottime dei sottoproblemi (**sottostruttura ottima**);
- un algoritmo ricorsivo per lo stesso problema richiederebbe di risolvere più volte alcuni sottoproblemi elevando troppo il costo computazionale (**sottoproblemi sovrapposti**);
- i sottoproblemi possono essere descritti da un numero limitato di parametri, cioè da uno **stato**.

Esempi di applicazione

- Sequenza DNA può essere rappresentata come una stringa con caratteri in $\{A, C, G, T\}$.
- Date due sequenze, il loro confronto può dare indicazioni su quanto sono correlati i due organismi delle due sequenze.
- Ci sono diversi modi di confrontare due sequenze DNA:
 - verificare se una è sottostringa dell'altra,
 - determinare quante modifiche si devono fare in una per farla diventare uguale all'altra (esercizio di questa lezione),
 - trovare una sottosequenza comune a entrambe in cui le basi si presentano nello stesso ordine (anche se non sono consecutive).
- In questa lezione si presenta come determinare una sottosequenza comune di lunghezza massima e come determinare una corrispondenza tra due sequenze in modo approssimato usando la tecnica della programmazione dinamica.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Sottosequenza

Data una sequenza di n oggetti (caratteri) $\langle a_1, \dots, a_n \rangle$, una **sottosequenza** è un sottoinsieme ordinato di questi oggetti $\langle a_{i_1}, a_{i_2}, \dots, a_{i_k} \rangle$, dove la sequenza d'indici $\langle i_1, i_2, \dots, i_k \rangle$ è strettamente crescente.

Esempio 1

Data la sequenza $X = \langle A, B, C, B, D, A, B \rangle$, la sequenza $Z = \langle B, C, D, B \rangle$ è sottosequenza di X perché gli indici dei suoi caratteri in X formano la sequenza crescente $\langle 2, 3, 5, 7 \rangle$.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Sottosequenza comune

Date due sequenze X e Y , una sequenza Z è **sottosequenza comune** di X e Y se è sottosequenza di entrambe.

Esempio 2 (CLRS)

Sia $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$.

La sequenza $\langle B, C, A \rangle$ è una sottosequenza comune come la sottosequenza $\langle B, C, B, A \rangle$.

La sottosequenza comune $\langle B, C, B, A \rangle$ è la **più lunga** possibile.

In inglese è chiamata *Longest Common Subsequence (LCS)*.

Esempi di applicazione

Massima sottosequenza comune (LCS)

PROBLEMA MASSIMA SOTTOSEQUENZA COMUNE (LCS)

INPUT: Date due sequenze X e Y sul medesimo alfabeto.

OUTPUT: Determinare la sottosequenza comune di lunghezza massima.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Fase 1: Caratterizzazione ricorsiva della struttura di una soluzione ottima.

Dato $X = \langle x_1, \dots, x_m \rangle$, sia $X_i = \langle x_1, \dots, x_i \rangle$ l' i -esimo **prefisso** di X .

Teorema 1

Dati $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$ e una LCS $Z = \langle z_1, z_2, \dots, z_k \rangle$:

- ❶ *se $x_m = y_n$, allora $z_k = x_m$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1} altrimenti*
- ❷ *se $x_m \neq y_n$ e $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y*
- ❸ *se $x_m \neq y_n$ e $z_k \neq y_n$, allora Z è una LCS di X e Y_{n-1} .*

Dimostrazione

Punto 1. Se $x_m = y_n$ ma Z non termina con tale carattere, allora potremmo sostituire l'ultimo carattere di Z con x_m ottenendo una LCS che termina con $x_m = y_n$. Tolto l'ultimo carattere, la parte rimanente è una LCS di X_{m-1} e Y_{n-1} . (continua...)

Esempi di applicazione

Massima sottosequenza comune (LCS)

continuazione dimostrazione.

Punto 2. Se $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y . Infatti se esistesse una sottosequenza comune W di X_{m-1} e Y di lunghezza $> k$, allora W sarebbe anche sottosequenza comune di X e Y contraddicendo l'ipotesi che Z sia una LCS di X e Y .

Punto 3. Dimostrazione simmetrica a quella del punto 2. □

- Quindi se una soluzione ottima ha il k -esimo carattere diverso dall'ultimo carattere di X o di Y , essa è soluzione ottima anche del sottoproblema in cui l'ultimo carattere di X o di Y non è presente.
- La soluzione ottima contiene le soluzioni ottime dei sottoproblemi.
- Una soluzione ricorsiva è quindi possibile!

Esempi di applicazione

Massima sottosequenza comune (LCS)

Fase 2: Definizione ricorsiva del valore di una soluzione ottima.

- Se $x_m = y_n$, il teorema assicura che si deve trovare una LCS per X_{m-1} e Y_{n-1} per poi accodare x_m per avere una LCS al problema originale.
- Se $x_m \neq y_n$, si devono risolvere due sottoproblemi e scegliere la soluzione che offre la LCS più lunga:
 - Trovare una LCS per X_{m-1} e Y ,
 - Trovare una LCS per X e Y_{n-1} .
- Si noti che, nel caso peggiore, in entrambi i rami ricorsivi si arriverà a risolvere il sottoproblema di determinare una LCS per X_{m-1} e Y_{n-1} (sottoproblema che verrà risolto 2 volte!).

Esempi di applicazione

Massima sottosequenza comune (LCS)

- Si può formulare in modo più astratto la soluzione ottima che si sta cercando.
- Sia $c[i, j]$ la lunghezza di una LCS delle sequenze X_i e Y_j .
- La soluzione del problema iniziale è $c[m, n]$.
- Il valore $c[i, j]$ è definito in modo ricorsivo

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i - 1, j - 1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max\{c[i, j - 1], c[i - 1, j]\} & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Esempi di applicazione

Massima sottosequenza comune (LCS): approccio *divide et impera*

- Se si resolvesse con il *divide et impera*, si otterrebbe un algoritmo corretto? **Sì!**
- Un algoritmo *divide et impera* risolverebbe il problema in modo ricorsivo resolvendo o il sottoproblema unico (quando $x_i = y_j$) o i due sottoproblemi in cui uno dei due input viene accorciato (per poi scegliere la soluzione migliore).
- Il caso peggiore è quando le due sequenze non hanno una sottosequenza comune non vuota (oppure ne hanno solo una di lunghezza 1).

Esempi di applicazione

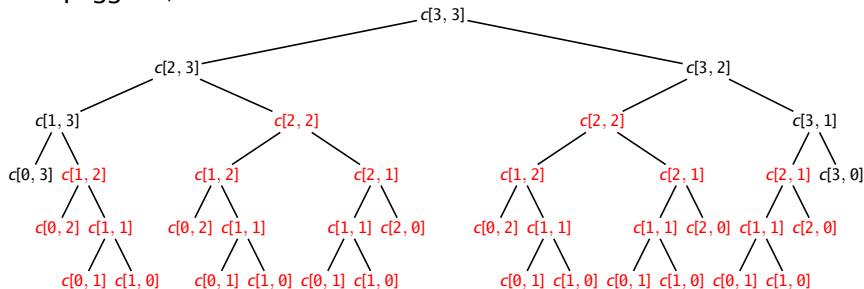
Massima sottosequenza comune (LCS): approccio *divide et impera*

- Nel caso peggiore, molti sottoproblemi sarebbero risolti più volte a causa delle chiamate ricorsive.
- Se si rappresentano le chiamate ricorsive in un albero, si ha subito un'idea del problema della perdita di tempo nel risolvere più volte gli stessi sottoproblemi.

Esempi di applicazione

Massima sottosequenza comune (LCS): approccio *divide et impera*

Per esempio, se le stringhe d'input hanno lunghezza $i = 3, j = 3$, l'albero delle chiamate ricorsive diventa (nel caso peggiore):



Nel caso peggiore, il numero $T(i, j)$ di chiamate soddisfa $T(i, j) = 1 + T(i - 1, j) + T(i, j - 1)$, con $T(0, j) = T(i, 0) = 1$.

Quindi $T(i, j) = 2^{\binom{i+j}{i}} - 1$, valore esponenziale quando i e j crescono insieme.

Nell'albero, $T(3, 3) = 2^{\binom{6}{3}} - 1 = 2 \cdot 20 - 1 = 39$.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Fase 3: Calcolo del valore di una soluzione ottima secondo uno schema bottom-up.

- In realtà i sottoproblemi distinti sono solo $\Theta(nm)$, perché ogni sottoproblema è identificato da una coppia di prefissi (X_i, Y_j) .
- Quindi se si risolvono ciascuno di questi sottoproblemi una e una sola volta, la combinazione di essi mediante l'equazione ricorsiva permette di determinare la soluzione in modo più efficiente.
- Quindi l'approccio è risolvere i sottoproblemi partendo dal più piccolo al più grande (approccio bottom-up) riusando i risultati ottenuti.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Fase 4: Costruzione della soluzione ottima:

- Cosa si deve aggiungere per risolvere la versione costruttiva?
- Si deve aggiungere una struttura ausiliaria che tenga traccia, per ogni soluzione parziale, di quale sottoproblema ha permesso di determinare la soluzione parziale.
- In questo modo è poi possibile ricostruire anche la struttura della soluzione ottima, non solo il valore $c[m, n]$.

Esempi di applicazione

Massima sottosequenza comune (LCS)

Algoritmo: LCS-LENGTH(X, Y)

Input: $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$

1: Siano $b[1 : m, 1 : n]$ e $c[0 : m, 0 : n]$ due nuove tabelle inizializzate a 0

// b serve per costruire la LCS Z

2: **foreach** $1 \leq i \leq m$ **do**

3: **foreach** $1 \leq j \leq n$ **do**

4: **if** $x_i = y_j$ **then**

5: $c[i, j] = c[i - 1, j - 1] + 1$

6: $b[i, j] = \nwarrow$

7: **continue**

8: **if** $c[i - 1, j] \geq c[i, j - 1]$ **then**

9: $c[i, j] = c[i - 1, j]$

10: $b[i, j] = \uparrow$

11: **else**

12: $c[i, j] = c[i, j - 1]$

13: $b[i, j] = \leftarrow$

14: **return** c e b

Il tempo di computazione è $\Theta(nm)$.

Esempi di applicazione

Massima sottosequenza comune (LCS)

- $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$
- LCS determinata da PRINT-LCS $\langle B, C, B, A \rangle$

		j	0	1	2	3	4	5	6
i	y_j	B	D	C	A	B	A		
0	x_i	0	0	0	0	0	0	0	
1	A	0	↑	↑	↑	↖1	←1	↖1	
2	B	0	↖1	←1	←1	↑1	↖2	←2	
3	C	0	↑1	↑1	↖2	←2	↑2	↑2	
4	B	0	↖1	↑1	↑2	↑2	↖3	←3	
5	D	0	↑1	↖2	↑2	↑2	↑3	↑3	
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4	
7	B	0	↖1	↑2	↑2	↑3	↖4	↑4	

Esempi di applicazione

Massima sottosequenza comune (LCS)

- La tabella b determinata da LCS-LENGTH può essere usata per costruire rapidamente una LCS delle due sequenze in input.
- PRINT-LCS è una procedura ricorsiva (perché più semplice e comunque efficiente) che determina la LCS una volta invocata come PRINT-LCS($b, X, |X|, |Y|$)

Algoritmo: PRINT-LCS(b, X, i, j)

Input: b = matrice delle mosse determinata da LCS-LENGTH; $X = \langle x_1, x_2, \dots, x_m \rangle$; i e j indici correnti nella ricorsione.

```
1: if  $i == 0$  or  $j == 0$  then return
2: if  $b[i, j] == \nwarrow$  then
3:   PRINT-LCS( $b, X, i - 1, j - 1$ )
4:   print  $x_i$ 
5:   return
6: if  $b[i, j] == \uparrow$  then
7:   PRINT-LCS( $b, X, i - 1, j$ )
8: else
9:   PRINT-LCS( $b, X, i, j - 1$ )
```

Si deriva facilmente che il tempo di computazione è $O(n + m)$.

Esempi di applicazione

Massima sottosequenza crescente

Sottosequenza crescente

Una **sottosequenza** $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ è **crescente** quando gli elementi sono confrontabili e sono in ordine strettamente crescente: $a_{i_1} < a_{i_2} < \dots < a_{i_k}$.

PROBLEMA MASSIMA SOTTOSEQUENZA CRESCENTE

INPUT: Una sequenza di interi $a_1, \dots, a_n$.

OUTPUT: Determinare la sottosequenza crescente di lunghezza massima.

Esempio: 5, 2, 8, 6, 3, 6, 9, 7 $\Rightarrow$ 2, 3, 6, 9

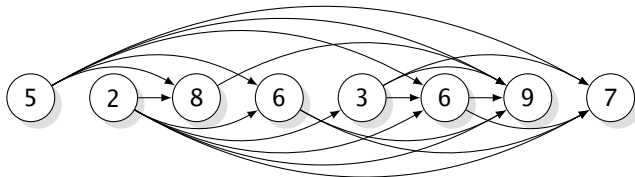


Esempi di applicazione

Massima sottosequenza crescente

Fase 1: Caratterizzazione ricorsiva della struttura di una soluzione ottima.

- Data un'istanza, si rappresentano graficamente le relazioni fra gli elementi rispetto alla soluzione richiesta:



- Il grafo è un DAG, perché ogni arco (i, j) soddisfa $i < j$, quindi non possono esistere cicli.
- C'è una corrispondenza biunivoca tra sottosequenze crescenti e cammini nel DAG.
- Trovare la massima sottosequenza crescente è equivalente a trovare il massimo cammino nel DAG.

Esempi di applicazione

Massima sottosequenza crescente

Fase 2: Definizione ricorsiva del **valore** di una soluzione ottima.

- Sia $L(j)$ la lunghezza del cammino più lungo che termina in j :

$$L(j) = 1 + \max (\{\emptyset\} \cup \{L(i) \mid (i, j) \in E\})$$

- Se si determinano $L(1), \dots, L(n)$, il **valore della soluzione ottima** è dato dal valore $L(j)$ massimo.
- $L(1), \dots, L(n)$ sono le sottoistanze da risolvere.

Fase 3: Calcolo del valore di una soluzione ottima secondo uno schema bottom-up.

- Se si risolvono in ordine partendo da $L(1)$, si determinano tutti i valori senza dover ricalcolare nessuno di essi.
- Per risolvere occorre conoscere i predecessori per ciascun j :
come si fa? quanto costa?

Fase 4: Costruzione della soluzione ottima: Cosa si deve aggiungere per risolvere la versione costruttiva?

Esempi di applicazione

Massima sottosequenza crescente

Versione valutativa:

Procedura MSCRESC-V(A)

Input: Una sequenza di interi $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: La lunghezza della massima sottosequenza crescente di A .

1: Si costruisca il DAG $G = (V, E)$ da A

2: L array di n interi

3: $ottimo = -\infty$

4: **foreach** $1 \leq j \leq n$ **do**

5: $max = 0$

6: **foreach** $(i \mid (i, j) \in E)$ **do** // $(i, j) \in E$ è da risolvere!

7: **if** $(L[i] > max)$ **then** $max = L[i]$

8: $L[j] = 1 + max$

9: **if** $(L[j] > ottimo)$ **then** $ottimo = L[j]$

10: **return** $ottimo$

// Soluzioni parziali
// Valore soluzione ottima

Esempi di applicazione

Massima sottosequenza crescente

Versione costruttiva:

Procedura MSCRESC(A)

Input: Una sequenza di interi $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: La lunghezza della massima sottosequenza crescente di A .

```
1: Si costruisca il DAG  $G^T = (V, E^T)$  da  $A$ 
2:  $L = \text{new int}[n]$ ; ottimo =  $-\infty$ 
3:  $\text{int}[]$  pred
4:  $i\text{Fine} = 0$ 
5: foreach  $1 \leq j \leq n$  do
6:    $\text{pred}[j] = \perp$ 
7:    $\text{max} = 0$ 
8:   foreach  $(i \mid (j, i) \in E^T)$  do //  $(i, j) \in E$  risolto!
9:     if  $(L[i] > \text{max})$  then  $\text{max} = L[i]$ ;  $\text{pred}[j] = i$ 
10:    $L[j] = 1 + \text{max}$ 
11:   if  $(L[j] > \text{ottimo})$  then  $\text{ottimo} = L[j]$ ;  $i\text{Fine} = j$ 
12: return REVERSEPRINT( $A, \text{pred}, i\text{Fine}$ )
```

// Mantiene i predecessori
// Indice nodo finale sequenza ottima

Esempi di applicazione

Massima sottosequenza crescente: complessità

Correttezza

Per induzione su j : quando si calcola $L(j)$, tutti i predecessori $i < j$ sono già stati risolti. Quindi $L(j)$ contiene la lunghezza del cammino crescente più lungo che termina in j .

Complessità

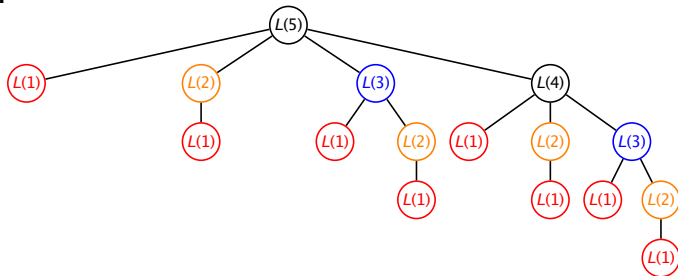
- Costruzione grafo trasposto richiede tempo $O(n^2)$ in quanto per ciascun elemento si deve inserire un arco orientato verso il nodo che rappresenta l'elemento in ciascuna lista di adiacenza dei nodi che rappresentano elementi inferiori del corrente.
- Il calcolo di $L(j)$ richiede tempo proporzionale al grado del nodo j . Sommando su tutti i nodi: $O(|E|)$. $|E| \leq (n-1)n/2$
- Totale: $O(n^2)$.

Esempi di applicazione

Massima sottosequenza crescente: approccio *divide et impera*

Se si risolvesse con il *divide et impera*, non si ottiene un algoritmo più semplice?

- La soluzione ricorsiva è sempre la medesima $L(j) = 1 + \max \{L(i) \mid (i, j) \in E\}$.
- Se il DAG contiene tutti gli archi possibili, la soluzione diventa $L(j) = 1 + \max \{L(1), L(2), \dots, L(j-1)\}$. Per esempio, se $j = 5$, l'albero delle chiamate ricorsive diventa:



Numero di chiamate ricorsive generate dalla chiamata iniziale: $2^{j-1} - 1$. Contando anche la chiamata iniziale si ottengono 2^{j-1} chiamate.

String matching approssimato: idea

Esempio 3

P è presente con 3 errori:

$T = \text{It's snowy}$ e $P = \text{sunny}$

It's s now
sunn y

In questo caso il pattern ha più caratteri coincidenti ma gli errori rimangono ancora 3.

Esempi di applicazione

String matching approssimato: idea

Esempio 5

Se $T = \text{It's } \square \text{snouwy}$ e $P = \text{snowy}$ (T ha una parola errata)

Una ricerca senza considerare i caratteri mancanti porta a una corrispondenza con 3-errori:

`I t ' s □ s n o u w y`

`s n o w y`

Se si considera che 'u' non è presente in P , si può ignorare 'u' (1 errore) e allineare il carattere corrente di P con il carattere precedente a 'u' in T

(gap in P): `I t ' s □ s n o u w y`
 `s n o _ w y`

Esempi di applicazione

String matching approssimato: idea

Esempio 6

Se $T = \text{It's } \square \text{snowy}$ e $P = \text{snowy}$ (T ha una parola errata)

Una ricerca senza considerare i caratteri mancanti porta a una corrispondenza con 2-errori:

It's $\square$ s o w y

s n o w y

Se si considera che 'n' non è presente in T , si può ignorare 'n' (1 errore) e allineare il carattere corrente di T con il carattere precedente a 'n' in P

(gap in T): It's $\square$ s o w y
 s n o w y

Esempi di applicazione

String matching approssimato: idea

- L'idea quindi è cercare una corrispondenza approssimata scegliendo la migliore fra tre possibilità:
 - 1 una data considerando il pattern e il testo così come sono,
 - 2 una assumendo che al pattern manchi un carattere (spostarsi solo nel testo), e
 - 3 una assumendo che al testo manchi un carattere (spostarsi solo nel pattern).

Esempi di applicazione

String matching approssimato

Definizione 1 (Stringhe k -approssimate)

Data una stringa T , **testo**, di lunghezza n e una stringa P , **pattern**, di $m < n$ caratteri, un'**occorrenza k -approssimata** di P in T è un allineamento di P con una sottostringa di T in cui compaiono k **errori** del tipo:

- 1 **sostituzione**: i corrispondenti caratteri sono diversi;
- 2 **gap in P** : un carattere di T non è allineato con un carattere di P ;
- 3 **gap in T** : un carattere di P non è allineato con un carattere di T ;

Esempi di applicazione

String matching approssimato

PROBLEMA STRING MATCHING APPROSSIMATO

INPUT: Un testo T di lunghezza n e un pattern P di lunghezza $m < n$.

OUTPUT: Determinare un'occorrenza k -approssimata minima (rispetto a k) di P in T .

Nel libro di testo, questo problema è risolto con tecniche più avanzate. Quindi si propone di risolverlo con la programmazione dinamica.

Nota!

Si verifica che $k \leq m$.

Se si ottiene $k = 0 \Rightarrow$ istanza positiva di STRING MATCHING.

Il problema EDIT DISTANCE è una generalizzazione in cui i **gap** sono visti come operazioni di **inserimento/cancellazione** da fare su T per trasformarlo in P . Per questo problema non ci sono vincoli per n, m .

Esempi di applicazione

String matching approssimato

Fase 1: Caratterizzazione ricorsiva della struttura di una soluzione ottima.

- Si supponga di avere un'occorrenza k -approssimata K **ottima** di P in T di lunghezza l .

Esempio 7

$T = \text{It's_snowy}$

$P = \text{snoww} \Rightarrow$ allineamento $\text{snow}\textcolor{red}{y}$ con $\text{snow}\textcolor{red}{w}$ (sostituzione)

$P = \text{snowwy} \Rightarrow$ allineamento $\text{snow}\textcolor{teal}{-}y$ con snowwy (gap in T)

$P = \text{sowy} \Rightarrow$ allineamento snowy con $s\textcolor{teal}{-}owy$ (gap in P)

K è un'occorrenza 1-approssimata ottima.

- Verifichiamo se è possibile scrivere K come composizione di una soluzione ottima di un sottoproblema.

Esempi di applicazione

String matching approssimato

Fase 1: Caratterizzazione ricorsiva della struttura di una soluzione ottima.

- Supponiamo di conoscere una soluzione ottima per allineare il sottopattern $P_i = \langle p_1, \dots, p_i \rangle$ con una sottostringa di T , $T_j = \langle t_1, \dots, t_j \rangle$.
- L'ultima scelta dell'allineamento può essere solo una delle tre seguenti:
 - 1 allineare p_i con t_j : costo 0 se $p_i = t_j$, costo 1 altrimenti;
 - 2 allineare p_i con un gap in T : costo 1;
 - 3 allineare t_j con un gap in P : costo 1.

Esempi di applicazione

String matching approssimato

- Se si allinea p_i con t_j , il sottoproblema rimanente è allineare $P_{i-1} = \langle p_1, \dots, p_{i-1} \rangle$ con una sottostringa di $T_{j-1} = \langle t_1, \dots, t_{j-1} \rangle$.
- Se si allinea p_i con un gap in T , il sottoproblema rimanente è allineare $P_{i-1} = \langle p_1, \dots, p_{i-1} \rangle$ con una sottostringa di $T_j = \langle t_1, \dots, t_j \rangle$.
- Se si allinea t_j con un gap in P , il sottoproblema rimanente è allineare $P_i = \langle p_1, \dots, p_i \rangle$ con una sottostringa di $T_{j-1} = \langle t_1, \dots, t_{j-1} \rangle$.
- In tutti e tre i casi, se la soluzione del sottoproblema non fosse ottima, sostituendola con una migliore si migliorerebbe anche la soluzione complessiva.

Esempi di applicazione

String matching approssimato

Fase 2: Definizione ricorsiva del **valore** di una soluzione ottima.

- Il **valore della soluzione ottima** può essere espresso tramite $K[i, j] \stackrel{\text{def}}{=}$ minimo numero di errori nell'approssimare P_i con una sottostringa di T_j che termina in posizione j .
- Il valore ottimo k finale è il minimo valore che si trova nell'ultima riga: $k = \min_j K[m, j]$.
- Viste le relazioni della sotto struttura ottima:
 - Caso 1: Nessun errore: $K[i, j] = K[i - 1, j - 1]$
 - Caso 2: c'è un errore: $K[i, j] = 1 + K[i - 1, j - 1]$
 - Caso 3: c'è un errore dato dal gap in T : $K[i, j] = 1 + K[i - 1, j]$.
 - Caso 4: c'è un errore dato dal gap in P : $K[i, j] = 1 + K[i, j - 1]$.
- Se $p_i == t_j$ (caso 1), il valore di $K[i, j] = K[i - 1, j - 1]$ senza nessun dubbio.
- Se $p_i \neq t_j$ (casi 2-4), si devono risolvere i 3 sottoproblemi, prendere la soluzione minima fra i 3 e aggiungere 1.

Esempi di applicazione

String matching approssimato

- Si può scrivere il calcolo per $K[i, j]$ in un'unica espressione:

$$K[i, j] = \min\{\text{diff}(i, j) + K[i-1, j-1], 1 + K[i-1, j], 1 + K[i, j-1]\}$$

dove $\text{diff}(i, j) \stackrel{\text{def}}{=} 0$ se $P[i] == T[j]$, 1 altrimenti.

Fase 3: Calcolo del valore di una soluzione ottima secondo uno schema bottom-up.

- Le soluzioni $K[i, j]$ formano una tabella bidimensionale.
- Tale tabella può essere completata riga per riga (da 1 a m) o colonna per colonna (da 1 a n).
- Rimane solo da definire la base della ricorsione: $K[\cdot, 0]$ e $K[0, \cdot]$:
 - $K[i, 0] = i$ in quanto P ha i caratteri in più di ε (ε è la stringa vuota);
 - $K[0, j] = 0$ perché il pattern vuoto si allinea con costo 0 a qualunque prefisso del testo.

Esempi di applicazione

String matching approssimato

Esempio 8

$T = \text{"It's snowy"}$ e $P = \text{"sunny"}$

	ε	I	t	'	s		s	n	o	w	y
ε	0	0	0	0	0	0	0	0	0	0	0
s	1	1	1	1	0	1	0	1	1	1	1
u	2	2	2	2	1	1	1	1	2	2	2
n	3	3	3	3	2	2	2	1	2	3	3
n	4	4	4	4	3	3	3	2	2	3	4
y	5	5	5	5	4	4	4	3	3	3	3

Istanza per
STRING MATCHING APPROSSIMATO

Esempi di applicazione

String matching approssimato: pseudocodifica

Algoritmo: SMA(T, P)

Input: T è il testo, P è il pattern

Output: (numero di errori; posizione ultimo carattere di P in T)

```
1:  $n = |T|$ ;  $m = |P|$ 
2:  $K$  = matrice di dimensione  $(m + 1) \times (n + 1)$ 
3: foreach  $0 \leq i \leq n$  do  $K[0, i] := 0$ 
4: foreach  $0 \leq i \leq m$  do  $K[i, 0] := i$ 
5: foreach  $1 \leq i \leq m$  do
6:   foreach  $1 \leq j \leq n$  do
7:      $\text{diff} := (T[j] == P[i]) ? 0 : 1$ 
8:      $K[i, j] := \min\{\text{diff} + K[i - 1, j - 1], \text{ } 1 + K[i - 1, j], \text{ } 1 + K[i, j - 1]\}$ 
9:  $\text{Min} := +\infty$ ;  $\text{pos} := -1$  // Cerca la soluzione ottima nell'ultima riga
10: foreach  $1 \leq j \leq n$  do
11:   if ( $K[m, j] < \text{Min}$ ) then
12:      $\text{Min} := K[m, j]$ ;  $\text{pos} := j$ 
13: return ( $\text{Min}, \text{pos}$ )
```

Esempi di applicazione

String matching approssimato: complessità

Correttezza

Per induzione sull'ordine di riempimento della tabella: ogni cella $K[i, j]$ usa solo celle già calcolate e considera tutte le possibili ultime scelte dell'allineamento. Quindi $K[i, j]$ è il minimo numero di errori per il sottoproblema corrispondente.

Complessità

- La complessità è data dal costo di determinazione della tabella: $\Theta(nm)$.
- Si può completare SMA() in modo che restituisca anche la stringa approssimata sempre al medesimo costo.
- Se il problema esplicita una soglia k , esistono algoritmi più avanzati, come Landau–Vishkin, che sfruttano tale vincolo per ottenere complessità $\Theta(kn)$.

Fase 4: Costruzione della soluzione ottima: lasciato per esercizio.

Esempi di applicazione

Problema dello ZAINO

PROBLEMA ZAINO

INPUT: Un insieme di n oggetti caratterizzati ciascuno da un valore v_i e da un peso p_i , entrambi interi positivi; un intero positivo P , portata massima dello zaino.

OUTPUT: Determinare un vettore $x \in \{0, 1\}^n$ tale che il valore totale $\sum_{i=1}^n x_i v_i$ sia massimo e il peso totale $\sum_{i=1}^n x_i p_i \leq P$.

Nota!

Questa versione del problema è anche chiamata ZAINO 0-1 perché ogni oggetto o si prende o si lascia. Esistono anche le versioni ZAINO frazionario e ZAINO con ripetizione. Qui si presenta la soluzione per la versione classica lasciando quella per la versione con ripetizione come esercizio.

Esempi di applicazione

Problema dello ZAINO

Fase 1: Caratterizzazione ricorsiva della struttura di una soluzione ottima.

- Dimostrazione con tecnica **taglia e incolla**.
- Si consideri una soluzione ottima del sottoproblema con i primi j oggetti e capacità w .
- Se l'oggetto j non è nella soluzione, allora la soluzione è ottima per il sottoproblema con i primi $j - 1$ oggetti e capacità w .
- Se l'oggetto j è nella soluzione, tolto tale oggetto rimane una soluzione ottima per i primi $j - 1$ oggetti e capacità $w - p_j$.
- Se la soluzione rimanente non fosse ottima, sostituendola con una migliore si otterrebbe una soluzione migliore anche per il problema originale.

Esempi di applicazione

Problema dello ZAINO

Esempio 9

Istanza:

$$x = [x_1, x_2, x_3, x_4],$$

$$v = [6, 13, 4, 4],$$

$$p = [3, 13, 5, 3],$$

$$P = 13.$$

Zaino ottimo:

$$Z = \{x_1, x_3, x_4\}$$

$$\text{valore} = 6 + 4 + 4 = 14$$

$$\text{peso} = 3 + 5 + 3 = 11.$$

Se si toglie l'elemento x_4 dallo zaino ottimo Z , lo zaino rimanente $Z' = \{x_1, x_3\}$ è necessariamente lo zaino ottimo del sottoproblema:

$$x' = [x_1, x_2, x_3],$$

$$v' = [6, 13, 4],$$

$$p' = [3, 13, 5],$$

$$P' = 13 - 3 = 10.$$

Esempi di applicazione

Problema dello ZAINO

Fase 2: Definizione ricorsiva del valore di una soluzione ottima.

- Sia $K[j, w] \stackrel{\text{def}}{=}$ il massimo valore ottenibile usando uno zaino di capacità w e i primi j oggetti $(1, \dots, j)$.
- Il problema chiede di determinare il $K[n, P]$.
- Si considera allora un generico $K[j, w]$ e lo si esprime in termini di $K[]$ con argomenti di dimensioni inferiori.
 - Dato l'elemento j si deve decidere se tenerlo o non tenerlo nello zaino.
 - Se $p_j > w$, l'oggetto j non può essere inserito e quindi $K[j, w] = K[j - 1, w]$.
 - Se $p_j \leq w$, una strada esplora quale valore finale di zaino si ottiene se si tiene:
 $K[j - 1, w - p_j] + v_j$;
 - L'altra esplora quale altro valore si ottiene se si scarta: $K[j - 1, w]$.
 - Si prende poi il massimo tra i due: $K[j, w] = \max\{K[j - 1, w - p_j] + v_j, K[j - 1, w]\}$.

Esempi di applicazione

Problema dello ZAINO

Fase 3: Calcolo del valore di una soluzione ottima secondo uno schema bottom-up.

- Le soluzioni $K[j, w]$ formano una tabella bidimensionale.
- Tale tabella può essere completata riga per riga (da 1 a n) o colonna per colonna (da 1 a P).
- Rimane solo da definire la base della ricorsione: $K[·, 0]$ e $K[0, ·]$:
 - $K[0, w] = 0$ in quanto per qualsiasi capacità w con 0 oggetti non si ha nessun valore;
 - $K[j, 0] = 0$ perché lo zaino non può contenere nulla.

Esempi di applicazione

Problema dello ZAINO

Esempio 10

Istanza:

$x = [x_1, x_2, x_3, x_4]$,

$v = [6, 13, 4, 4]$,

$p = [3, 13, 5, 3]$,

$P = 13$.

v	p	x	$\frac{w}{j}$	0	1	2	3	4	5	6	7	8	9	10	11	12	13
			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	3	x_1	1	0	0	0	6	6	6	6	6	6	6	6	6	6	6
13	13	x_2	2	0	0	0	6	6	6	6	6	6	6	6	6	6	13
4	5	x_3	3	0	0	0	6	6	6	6	6	10	10	10	10	10	13
4	3	x_4	4	0	0	0	6	6	6	10	10	10	10	10	14	14	14

Le celle colorate rappresentano le celle usate per determinare la soluzione ottima.

In blu le celle usate per determinare il valore della cella colorata nella riga successiva. Le celle evidenziate in giallo sono in corrispondenza agli elementi presenti nella soluzione ottima.

Esempi di applicazione

Problema dello ZAINO

Algoritmo: ZAINO(P, p, v)

Input: P portata dello zaino, p array dei pesi degli elementi, v array dei valori degli elementi.

Output: il valore ottimo dello zaino

```
1:  $n := |v|$ 
2: foreach  $0 \leq i \leq P$  do  $K[0, i] = 0$  // In Java non è necessario
3: foreach  $0 \leq i \leq n$  do  $K[i, 0] = 0$  // In Java non è necessario
4: foreach  $1 \leq j \leq n$  do
5:   foreach  $1 \leq w \leq P$  do
6:     if ( $p[j] > w$ ) then  $K[j, w] = K[j - 1, w]$  //  $j$  troppo pesante!
7:     else  $K[j, w] = \max(K[j - 1, w], K[j - 1, w - p[j]] + v[j])$ 
8: return  $K[n, P]$ 
```

Esempi di applicazione

Problema dello ZAINO

Correttezza

Per induzione su j e w : quando si calcola $K[j, w]$, le celle della riga precedente rappresentano già i valori ottimi dei sottoproblemi con i primi $j - 1$ oggetti. La ricorrenza confronta le uniche due possibilità: scartare o prendere l'oggetto j .

Complessità

Si verifica facilmente che è $\Theta(nP)$.

Tuttavia, $\Theta(nP)$ non è polinomiale nella dimensione dell'input quando P è codificato in binario.

Infatti, se k è il numero di bit necessari per rappresentare P , allora vale

$k = \Theta(\log P)$, cioè $P = \Theta(2^k)$.

Quindi il tempo $\Theta(nP)$ è polinomiale nel valore numerico di P , ma può essere esponenziale nella lunghezza della sua rappresentazione binaria.

Per questo motivo, la complessità $\Theta(nP)$ si dice **pseudo-polinomiale**.

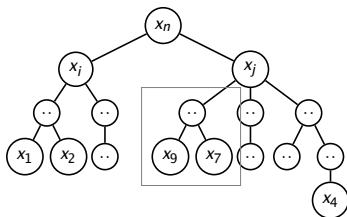
Fase 4: Costruzione della soluzione ottima: lasciato per esercizio.

- Aspetto cruciale per applicare la programmazione dinamica è caratterizzare in modo corretto come scomporre il problema in sottoproblemi.
- Servono creatività ed **esperienza**.
- L'esperienza di alcuni autori suggerisce però che esistono 4 modi con cui costruire i sottoproblemi che emergono più di altri:
 - 1 Input: $x_1, x_2, \dots, x_n$. Il sottoproblema considera l'input dal **primo elemento** fino all' i -esimo: $x_1, x_2, \dots, x_i$.
Numero dei sottoproblemi: $O(n)$.
 - 2 Input: $x_1, x_2, \dots, x_n$ e $y_1, y_2, \dots, y_m$. Il sottoproblema considera l'input dal **primo elemento** fino a un **certo** indice per entrambe le stringhe: $x_1, x_2, \dots, x_i$ e $y_1, y_2, \dots, y_j$.
Numero dei sottoproblemi: $O(nm)$.

Casi comuni

Sottoproblemi comuni

- 3 Input: $x_1, x_2, \dots, x_n$. Il sottoproblema considera una parte dell'input che non inizia dal primo elemento: $x_1, x_2, \dots, x_i, x_{i+1}, \dots, x_j$.
Numero dei sottoproblemi: $O(n^2)$. È sufficiente costruire una tabella $n \times n$ in cui le righe rappresentano i e le colonne j .
Che forma assume la matrice?
- 4 L'input è un albero con radice di ordine n . Il sottoproblema è un sotto-albero con radice.



Numero dei sottoproblemi: $O(n)$ perché n sono le possibili radici distinte.

- **Memoization**, termine senza 'r' coniato da Donald Michie nel 1968, indica una tecnica di ottimizzazione per accelerare l'esecuzione dei programmi:
Quando ci sono più chiamate di funzioni con medesimi parametri, si memorizzano i risultati delle chiamate per poi riusarli anziché eseguire nuovamente le funzioni.
- Si differenzia dal *caching* perché è esplicito nel contesto di un programma.
- Una funzione scritta usando la memoization usa una struttura di supporto per memorizzare i risultati delle chiamate in modo da poterli eventualmente riutilizzare nelle chiamate successive.
- La struttura può essere una hash table, un array o una matrice; la chiave è data dai parametri presenti nella chiamata della funzione.

Memoization

Esempio

K è una tabella hash **globale** con chiave (i, w) .

P , v e p sono variabili globali.

Procedura ZAINO(i, w)

// i indica l'elemento in esame. w la capacità dello zaino

- 1: **if** $K.CONTAINSKEY(i, w)$ **then return** $K.GET(i, w)$
 - 2: **if** $(i == 0)$ **or** $(w == 0)$ **then return** $\emptyset$
 - 3: **if** $(p[i] > w)$ **then** $k = \text{ZAINO}(i - 1, w)$ // i troppo pesante!
 - 4: **else** $k = \max(\text{ZAINO}(i - 1, w), \text{ZAINO}(i - 1, w - p[i]) + v[i])$
 - 5: $K.PUT((i, w), k)$
 - 6: **return** k
-

Complessità: $O(nP)$.

Medesima complessità dell'algoritmo costruito con la programmazione dinamica: le costanti moltiplicative sono però più grandi per l'overhead delle chiamate ricorsive.

Memoization

Quando si usa?

- Programmazione dinamica (Tabulazione bottom-up): risolve tutte le sottoistanze di un problema ricorsivo rispettando un ordine di dipendenza.
- *divide et impera*: risolve solo le sottoistanze di un problema ricorsivo utili per la soluzione finale ma risolve le sottoistanze comuni ogni volta che le “incontra”.
- Memoization: mantiene la struttura ricorsiva del *divide et impera*, ma riutilizza i risultati delle sottoistanze già incontrate.
- Un algoritmo con memoization potrebbe quindi essere più efficiente di una tabulazione bottom-up quando molte sottoistanze non sono necessarie.
- In generale l'overhead delle chiamate ricorsive può rendere la tabulazione più efficiente se risolve solo poche sottoistanze non necessarie.

- Studiare il capitolo 14 CLRS limitatamente alle sezioni 14.1 (nuovo esempio di applicazione), 14.3 (teoria) e 14.4 (ripasso LCS).
- Svolgere gli esercizi 14.1–3, 14.1–5, 14.3–2

Esercizio 1 (Massima sottosequenza comune (Esercizio 15.4-2 [CLRS]))

Implementare gli algoritmi `LCS-LENGTH` e `PRINT-LCS` in linguaggio Java senza il calcolo della matrice b . Si determinino i tempi di computazione con input di circa 1000, 10 000, 100 000 caratteri. (Quindi scrivere anche una procedura che costruisca stringhe casuali di lunghezza data usando un limitato numero di caratteri possibili (DNA)).

Esercizio 2

La complessità dell'algoritmo $\text{MSCRESC-V}()$ è $O(|E|)$. Esprimere tale complessità in funzione della lunghezza della sequenza in input e descrivere un esempio di caso peggiore.

Esercizio 3

Completare l'algoritmo `SMA()` in modo che in output stampi la sottosequenza di T per la quale la k -approssimazione è minima.